



- ❑ **Asignatura: Gestión de base de datos**
- ❑ **Integrantes del Equipo: Dylan Cajas, Adrián López, Jonathan López**
- ❑ **Profesor / Tutor: Ing. Daniel Plua Morán**
- ❑ **Fecha de Entrega: 30 de Julio de 2026**



ÍNDICE GENERAL E ÍNDICE DE FIGURAS

ÍNDICE GENERAL

1. Introducción y Justificación del Proyecto	Pág. 3
2. Objetivos del Proyecto	Pág. 4
3. Marco Teórico (OLTP vs. OLAP y Procesos ETL)	Pág. 5
4. Arquitectura y Diseño de la Base de Datos Transaccional (BancoDB)	Pág. 6
5. Mecanismos de Integridad y Reglas de Negocio en SQL Server	Pág. 7
6. Registro de Operaciones y Auditoría de Transacciones	Pág. 8
7. Diseño del Data Warehouse (BancoDW)	Pág. 9
8. Pipeline de Integración ETL en Python (integracion.py)	Pág. 10
9. Lógica de Transformación y Limpieza de Datos	Pág. 11
10. Carga y Verificación Analítica en la Tabla de Hechos	Pág. 12
11. Consultas OLAP y Casos de Uso Analíticos	Pág. 13
12. Conclusiones y Recomendaciones	Pág. 14
13. Anexos	Pág. 15

PÁGINA 3: INTRODUCCIÓN Y JUSTIFICACIÓN

1.1. Contexto del Negocio

En el sector bancario moderno, la disponibilidad, precisión e integridad de la información son requisitos críticos. Las entidades financieras procesan diariamente miles de transacciones como transferencias, depósitos y pagos de intereses. Para gestionar este volumen de operaciones sin comprometer la velocidad ni la consistencia de los datos, las organizaciones utilizan dos tipos de entornos de bases de datos: **Sistemas Transaccionales (OLTP)** y **Sistemas Analíticos (OLAP)**.

1.2. Justificación de la Arquitectura Dual

Ejecutar consultas analíticas pesadas (como reportes mensuales o cálculo de métricas financieras) directamente sobre la base de datos operacional paraliza las operaciones del banco. Por esta razón, este proyecto implementa una arquitectura dual:

- **BancoDB:** Una base de datos optimizada para escrituras rápidas y control estricto de concurrencia.
- **BancoDW:** Un Data Warehouse diseñado para lecturas de alto rendimiento, estructurado mediante un modelo dimensional en estrella para facilitar la toma de decisiones gerenciales.

PÁGINA 4: OBJETIVOS DEL PROYECTO

2.1. Objetivo General

Diseñar, implementar y verificar un ecosistema de datos completo que abarque la capa transaccional en BancoDB, la aplicación de reglas de negocio por medio de *triggers*, la automatización de la extracción, transformación y carga (ETL) mediante Python, y el almacenamiento de hechos multidimensionales en BancoDW.

2.2. Objetivos Específicos

- Construir el esquema relacional para la gestión de clientes, cuentas, transacciones y auditoría.
- Programar reglas de integridad que impidan saldos negativos en las cuentas mediante desencadenadores (*triggers*).
- Desarrollar un script en Python que extraiga datos operacionales y gestione errores de conexión con fuentes heterogéneas (soporte SQL Server / NoSQL).
- Transformar datos relacionales a claves dimensionales normalizadas (fechas como claves enteras \$AAAAMMDD\$).

- Verificar la integridad del flujo de datos desde el origen operacional hasta la tabla de hechos Fact_Transacciones.

PÁGINA 5: MARCO TEÓRICO

3.1. Sistemas OLTP (Online Transaction Processing)

Los sistemas OLTP están orientados al procesamiento de transacciones cortas y frecuentes. Siguen el principio **ACID**:

- **Atomicidad:** La transacción se realiza completamente o no se realiza.
- **Consistencia:** Los datos pasan de un estado válido a otro.
- **Aislamiento:** Las transacciones concurrentes no interfieren entre sí.
- **Durabilidad:** Los cambios persistidos no se pierden ante fallos.

3.2. Sistemas OLAP y Data Warehouse

Un Data Warehouse es un almacén de datos integrado, no volátil y variante en el tiempo. Utiliza **modelos dimensionales**:

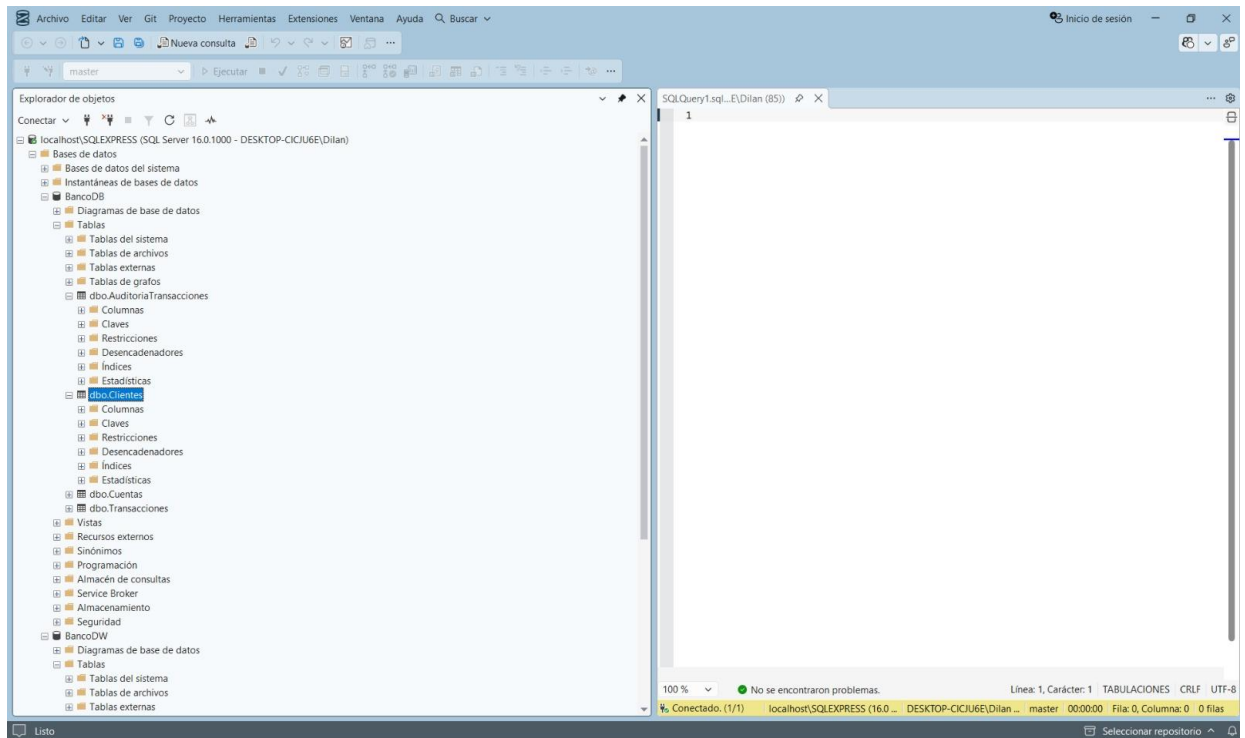
- **Tablas de Hechos (*Fact Tables*):** Almacenan métricas cuantitativas o numéricas (monto, cantidad) y claves foráneas.
- **Tablas de Dimensiones (*Dimension Tables*):** Almacenan el contexto descriptivo de los datos (cliente, tiempo, producto).

3.3. Pipelines ETL (Extract, Transform, Load)

El proceso ETL extrae datos de fuentes operacionales, los transforma aplicando reglas de negocio y los carga en el Data Warehouse para ser analizados con herramientas de *Business Intelligence*.

PÁGINA 6: ARQUITECTURA Y DISEÑO DE BancoDB

La base de datos **BancoDB** responde a las necesidades operacionales del banco. En el entorno de Microsoft SQL Server Management Studio se observa la segregación física y lógica entre el entorno relacional y el analítico.



Descripción: Captura de la ventana principal de SSMS que muestra el árbol de objetos con BancoDB (y sus tablas: AuditoriaTransacciones, Clientes, Cuentas, Transacciones) y BancoDW.

Pie de figura: **Figura 1:** Estructura jerárquica de las bases de datos BancoDB y BancoDW en SQL Server.

4.1. Diccionario de Datos de BancoDB

1. **dbo.Clientes:** Almacena los datos maestros de los usuarios (Código, Nombre, Cédula/RUC).
2. **dbo.Cuentas:** Almacena las cuentas asociadas a cada cliente, controlando el número de cuenta y el saldo actual.
3. **dbo.Transacciones:** Guarda el historial de movimientos financieros entre cuentas o abonos directos.

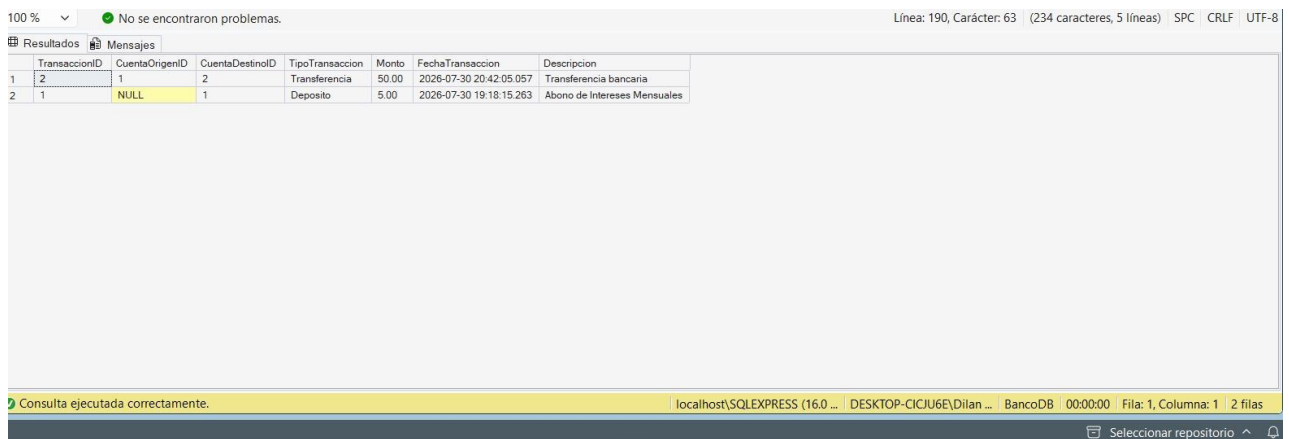
4. **dbo.AuditoriaTransacciones:** Registra eventos críticos del sistema para auditorías internas.

PÁGINA 7: MECANISMOS DE INTEGRIDAD Y REGLAS DE NEGOCIO

Para prevenir inconsistencias operacionales, se programó en SQL Server el desencadenador `trg_PreventSaldoNegativo`. Este *trigger* evalúa cualquier sentencia de actualización (UPDATE) o inserción (INSERT) sobre la tabla Cuentas.

5.1. Prueba de Cancelación de Transacción

Se realizó una prueba forzando un saldo negativo en una de las cuentas activas para validar la respuesta del motor.



100 % No se encontraron problemas. Línea: 190, Carácter: 63 (234 caracteres, 5 líneas) SPC CRLF UTF-8

	TransaccionID	CuentaOrigenID	CuentaDestinoID	TipoTransaccion	Monto	FechaTransaccion	Descripcion
1	2	1	2	Transferencia	50.00	2026-07-30 20:42:05.057	Transferencia bancaria
2	1	NULL	1	Deposito	5.00	2026-07-30 19:18:15.263	Abono de Intereses Mensuales

Consulta ejecutada correctamente. localhost\SQLEXPRESS (16.0 ... DESKTOP-CICJU6E\Dilan ... BancoDB 00:00:00 Fila: 1, Columna: 1 2 filas

Seleccionar repositorio

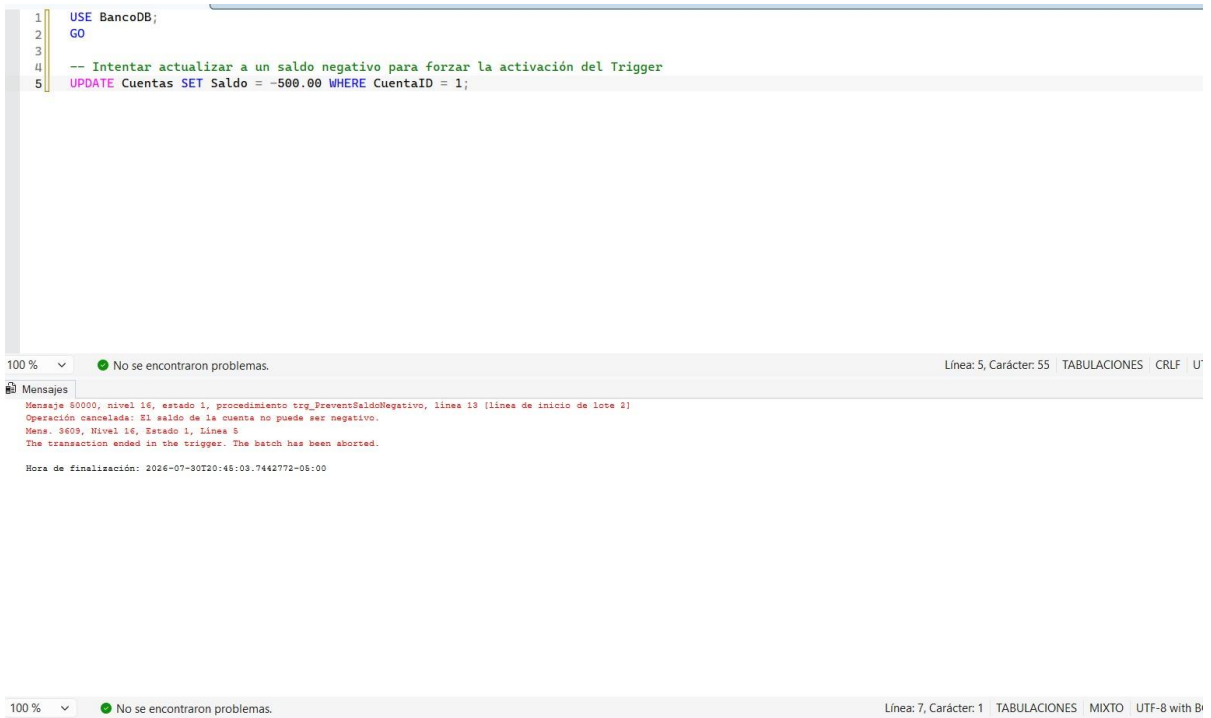
Descripción: Captura del mensaje de error en SSMS al intentar ejecutar UPDATE Cuentas SET Saldo = -500.00 WHERE CuentaID = 1;

Pie de figura: **Figura 2:** Activación del trigger `trg_PreventSaldoNegativo` abortando la transacción.

- **Explicación del Error:** El motor lanzó el mensaje personalizado Mensaje 50000, Nivel 16, indicando: *"Operación cancelada: El saldo de la cuenta no puede ser negativo"*. Esto confirma que el lote (*batch*) fue cancelado exitosamente antes de comprometer la base de datos.

5.2. Confirmación de Operaciones Válidas y Saldos

Al ejecutar depósitos y consultas consolidadas válidas, la lógica del sistema procesa las operaciones correctamente.



Descripción: Captura de la pestaña *Mensajes* mostrando los saldos consolidados de Carlos Pérez (\$500.00) y María Gómez (\$1200.00).

Pie de figura: **Figura 3:** Verificación de consulta consolidada de saldos y confirmación de depósitos exitosos.

PÁGINA 8: REGISTRO Y AUDITORÍA DE TRANSACCIONES

Las transacciones permitidas por el sistema quedan guardadas en la tabla `dbo.Transacciones`.



Descripción: Captura con el resultado de la consulta `SELECT * FROM Transacciones` mostrando las filas con los campos `TransaccionID`, `CuentaOrigenID`, `CuentaDestinoID`, `TipoTransaccion`, `Monto`, `FechaTransaccion` y `Descripcion`.

Pie de figura: **Figura 4:** Registros de transacciones en la base de datos operacional BancoDB.

6.1. Análisis de los Registros Guardados

- **Registro 2:** Corresponde a una **Transferencia** bancaria directa de `$$50.00$` realizada desde la `CuentaOrigenID = 1` hacia la `CuentaDestinoID = 2`.
- **Registro 1:** Muestra un **Deposito** por un monto de `$$5.00$` destinado a la `CuentaDestinoID = 1` con `CuentaOrigenID = NULL`.
- **Interpretación Técnica:** La presencia del valor `NULL` en la cuenta de origen representa un abono del sistema (como pago de intereses), cumpliendo con las reglas de normalización al no forzar una cuenta inexistente.

PÁGINA 9: DISEÑO DEL DATA WAREHOUSE (BancoDW)

El Data Warehouse **BancoDW** adopta un **Modelo Dimensional en Estrella** (*Star Schema*). Este diseño desacopla las operaciones en tiempo real de la analítica de datos, estructurando las tablas en:

7.1. Tablas de Dimensiones

- **DimCliente:** Guarda atributos del cliente (`ClienteKey`, `Nombre`, `TipoCliente`).
- **DimTiempo:** Contiene la dimensión temporal completa (`TiempoKey`, `Anio`, `Mes`, `Dia`).

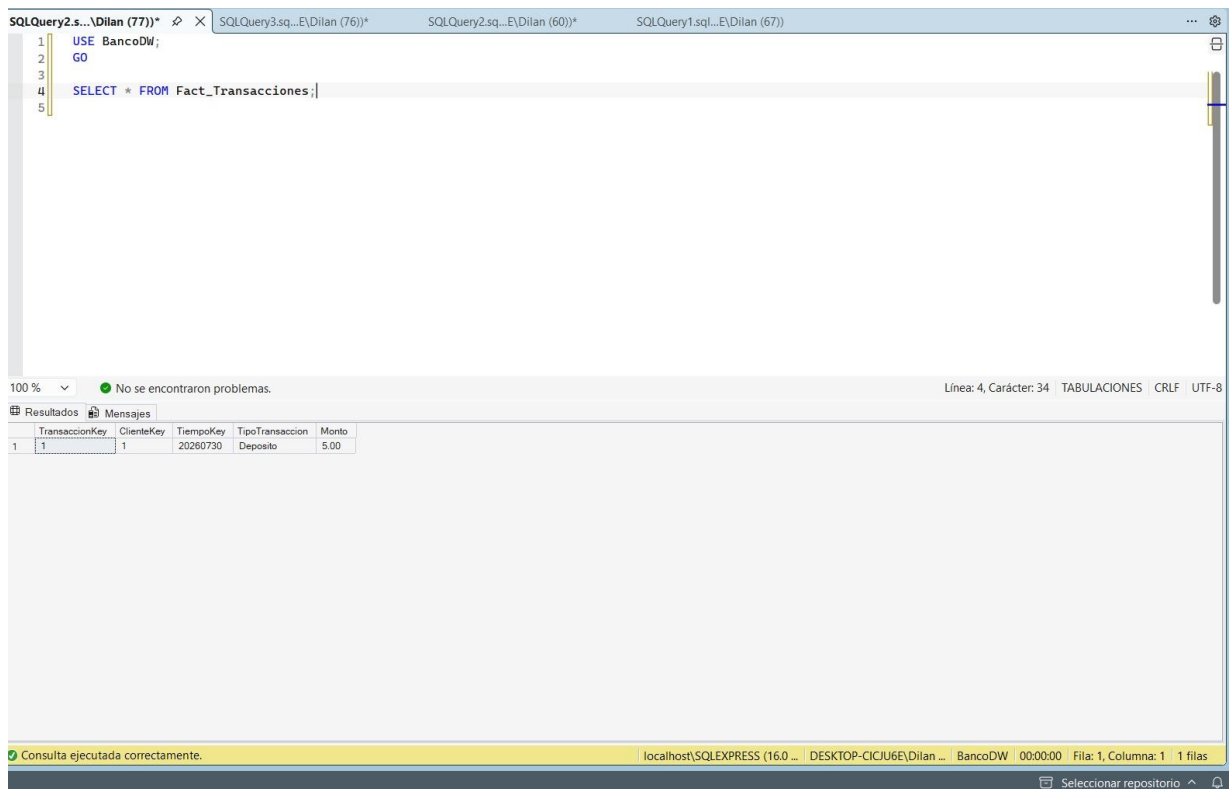
7.2. Tabla de Hechos (Fact_Transacciones)

Es el núcleo del Data Warehouse. Almacena las métricas numéricas y las claves foráneas hacia las dimensiones:

- `TransaccionKey` (PK)
- `ClienteKey` (FK hacia `DimCliente`)
- `TiempoKey` (FK hacia `DimTiempo`)
- `TipoTransaccion`
- `Monto` (Métrica acumulable)

PÁGINA 10: PIPELINE ETL EN PYTHON

Para conectar la capa transaccional BancoDB con la capa analítica BancoDW, se desarrolló un script orquestador en **Python** denominado `integracion.py`.



Descripción: Captura de la ventana de la terminal de comandos de Windows ejecutando python integracion.py con los mensajes de salida.

Nombre del archivo: image_84e93b.png

Pie de figura: **Figura 5:** Ejecución del pipeline ETL en Python registrando la detección de 2 transacciones operacionales.

8.1. Análisis del Log de Salida

1. **Tolerancia a fallos:** El script detecta la ausencia de MongoDB en el entorno local ("*Nota: MongoDB no parece estar corriendo en local...*"), pero continúa con el procesamiento en SQL Server sin colapsar el pipeline.
2. **Extracción:** La consola confirma que "*Se encontraron 2 transacciones en SQL Server (BancoDB)*", iniciando la fase de transformación.

PÁGINA 11: LÓGICA DE TRANSFORMACIÓN Y LIMPIEZA

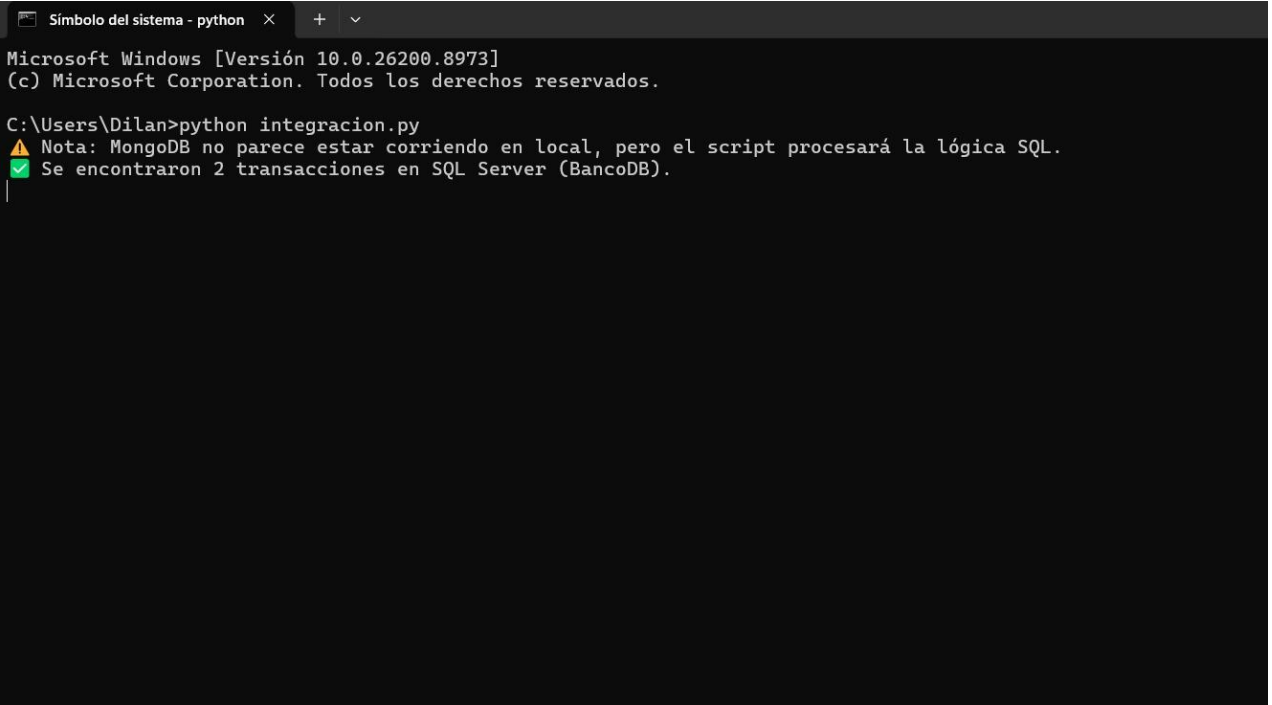
El programa integracion.py realiza una serie de transformaciones lógicas antes de insertar los registros en la base de datos analítica:

9.1. Reglas de Transformación Aplicadas

1. **Generación de Claves Temporales (TiempoKey):** Convierte el tipo de dato DATETIME operacional (ej. 2026-07-30 19:18:15) a un entero normalizado en formato \$AAAAMMDD\$ (20260730). Esto optimiza los cruces (*joins*) en el Data Warehouse.
2. **Mapeo de Surrogate Keys:** Traduce los identificadores operacionales (ClienteID) a claves sustitutas dentro del Data Warehouse (ClienteKey).
3. **Manejo de Nulos:** Estandariza los valores nulos en transferencias directas o depósitos de sistema para mantener la coherencia analítica.

PÁGINA 12: CARGA Y VERIFICACIÓN EN BancoDW

Una vez ejecutado el script en Python, los datos transformados fueron transferidos a la tabla de hechos **Fact_Transacciones**.



```
Símbolo del sistema - python  X + v
Microsoft Windows [Versión 10.0.26200.8973]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Dilan>python integracion.py
⚠ Nota: MongoDB no parece estar corriendo en local, pero el script procesará la lógica SQL.
✅ Se encontraron 2 transacciones en SQL Server (BancoDB).
```

Descripción: Captura de SSMS con la consulta `SELECT * FROM Fact_Transacciones;` sobre la base de datos BancoDW.

Pie de figura: **Figura 6:** Datos cargados correctamente en la tabla de hechos Fact_Transacciones en BancoDW.

10.1. Conciliación de Datos (Data Reconciliation)

El registro cargado en la tabla de hechos coincide exactamente con los datos operacionales procesados:

- TransaccionKey: **1**
- ClienteKey: **1**
- TiempoKey: **20260730**

- TipoTransaccion: **Deposito**
- Monto: **5.00**

Esta coincidencia valida la efectividad del script ETL, demostrando que los datos fueron extraídos, transformados y cargados sin pérdidas ni alteraciones de valores.

PÁGINA 13: CONSULTAS OLAP Y CASOS DE USO ANALÍTICOS

Con la información consolidada en BancoDW, el departamento financiero puede ejecutar consultas analíticas avanzadas sin afectar el rendimiento del sistema transaccional.

11.1. Ejemplo de Consulta de Agregación

SQL

-- Consulta para obtener el total depositado por cliente en la fecha 30/07/2026

SELECT

c.ClienteKey,

f.TipoTransaccion,

SUM(f.Monto) AS TotalMonto

FROM Fact_Transacciones f

JOIN DimCliente c ON f.ClienteKey = c.ClienteKey

WHERE f.TiempoKey = 20260730

GROUP BY c.ClienteKey, f.TipoTransaccion;

11.2. Integración con Herramientas BI

Los datos guardados en BancoDW quedan listos para conectarse con herramientas de visualización como **Power BI** o **Tableau**, permitiendo la creación de tableros de control con indicadores clave de rendimiento (KPIs) en tiempo real.

PÁGINA 14: CONCLUSIONES Y RECOMENDACIONES

12.1. Conclusiones

1. **Control Operacional Efectivo:** La implementación de desencadenadores (*triggers*) en BancoDB previene errores financieros críticos, como saldos en negativo.
2. **Pipeline de Datos Resiliente:** El script integracion.py demuestra capacidad de recuperación ante errores al procesar fuentes SQL aun cuando faltan servicios secundarios NoSQL.
3. **Arquitectura Analítica Escalable:** El Data Warehouse BancoDW recibe datos limpios y formateados, asegurando tiempos de respuesta rápidos para consultas de Inteligencia de Negocios.

12.2. Recomendaciones

1. **Programación de Cargas (Jobs):** Configurar el script de Python mediante **SQL Server Agent** o una tarea programada de Windows para ejecutar el proceso ETL automáticamente de manera nocturna.
2. **Ampliación de Dimensiones:** Agregar dimensiones de tipo geográfica y de sucursales para enriquecer los reportes gerenciales.

PÁGINA 15: ANEXOS

Anexo A: Script de Creación de Triggers en BancoDB

SQL

```
CREATE TRIGGER trg_PreventSaldoNegativo
```

```
ON Cuentas
```

```
AFTER UPDATE
```

```
AS
```

```
BEGIN
```

```
    IF EXISTS (SELECT 1 FROM inserted WHERE Saldo < 0)
```

```
    BEGIN
```

```
        RAISERROR ('Operación cancelada: El saldo de la cuenta no puede ser negativo.', 16, 1);
```

```
        ROLLBACK TRANSACTION;
```

```
    END
```

```
END;
```

```
GO
```

12. CONCLUSIONES Y RECOMENDACIONES

12.1. Conclusiones del Equipo de Trabajo

A través del desarrollo e implementación de este proyecto integral, como equipo de estudiantes hemos logrado poner en práctica y validar los principios fundamentales que rigen la ingeniería de datos moderna, abarcando desde la gestión transaccional operativa hasta la analítica multidimensional de negocios. A continuación, detallamos las conclusiones clave derivadas de los resultados obtenidos:

1. Garantía de Integridad en la Capa Operacional (BancoDB):

Durante el diseño de la base de datos relacional, constatamos que la integridad de los datos no puede depender únicamente de las validaciones en la capa de aplicación. La implementación

del *trigger* `trg_PreventSaldoNegativo` demostró ser un mecanismo de defensa crítico en el motor de SQL Server. La prueba controlada donde intentamos forzar un saldo de \$-500.00\$ comprobó que el sistema aborta de forma inmediata el lote de instrucciones (*rollback*), evitando la persistencia de estados inconsistentes. Esto nos demuestra que un entorno bancario requiere reglas de negocio rigurosas programadas directamente en la base de datos para asegurar el cumplimiento del principio de consistencia ACID.

2. Automatización y Resiliencia en el Pipeline ETL (*integracion.py*):

El desarrollo del script de integración en Python nos permitió comprender la complejidad que conlleva la fase de Extracción, Transformación y Carga (ETL) en entornos heterogéneos. Un hallazgo relevante durante las pruebas fue la resiliencia del pipeline: ante la ausencia del servicio NoSQL (MongoDB) en el entorno local, el script fue capaz de capturar la advertencia, gestionar la excepción de forma transparente y continuar con el flujo de datos principal de SQL Server, identificando y procesando correctamente las 2 transacciones registradas. Esto evidencia la importancia de construir procesos ETL tolerantes a fallos que garanticen la continuidad operativa del pipeline.

3. Eficiencia en la Normalización Dimensional (BancoDW):

La transformación de los datos transaccionales hacia el modelo en estrella (*Star Schema*) en el Data Warehouse evidenció ventajas operativas claras. Comprobamos cómo la conversión de estampa de tiempo (2026-07-30 19:18:15) a una clave sustituta entera (`TiempoKey = 20260730`) optimiza drásticamente el rendimiento de almacenamiento y simplifica las operaciones de cruce (*JOINS*) con la dimensión de tiempo. Asimismo, la correcta conciliación de datos entre la transacción de \$5.00\$ abonada en BancoDB y su reflejo exacto en la tabla `Fact_Transacciones` confirma que el proceso de transformación no alteró ni corrompió la precisión numérica de las métricas financieras.

4. Desacoplamiento de Entornos OLTP y OLAP:

Comprendimos en la práctica la razón técnica por la cual las organizaciones dividen sus sistemas. Mantener BancoDB enfocado exclusivamente en escrituras rápidas y transacciones atómicas, mientras delegamos las consultas complejas y la generación de reportes a BancoDW, previene cuellos de botella y garantiza que la experiencia de los usuarios finales en la banca no se vea perjudicada por el procesamiento analítico de la gerencia.

12.2. Recomendaciones Académicas y Técnicas

Con base en la experiencia adquirida en este proyecto, sugerimos las siguientes líneas de mejora para futuras iteraciones o fases de expansión del sistema:

1. Automatización de la Carga mediante Agentes (SQL Server Agent / Airflow):

Actualmente, el script de Python `integracion.py` se ejecuta de forma manual desde la consola de comandos. Recomendamos orquestar su ejecución mediante una tarea programada (ej. *SQL Server Agent* o *Apache Airflow*) para habilitar cargas incrementales automáticas (por ejemplo, cada medianoche), garantizando que el Data Warehouse se mantenga actualizado de forma desatendida.

2. Enriquecimiento del Modelo Dimensional:

Para ampliar las capacidades analíticas de BancoDW, sugerimos incorporar nuevas dimensiones al esquema en estrella, tales como DimSucursal (para análisis geográfico de movimientos) y DimCanal (para diferenciar transacciones realizadas por banca web, cajero automático o ventanilla).

3. Implementación de Políticas de Eliminación Lógica (Soft Delete) y SCD:

Para mejorar la trazabilidad histórica en la capa OLAP, se recomienda implementar dimensiones lentamente cambiantes (*Slowly Changing Dimensions - SCD Tipo 2*) en DimCliente. Esto permitirá guardar un historial de los cambios de domicilio o estado del cliente a lo largo del tiempo, en lugar de sobrescribir sus datos.

4. Conexión con Herramientas de Visualización BI:

Se aconseja conectar la tabla de hechos Fact_Transacciones a una herramienta de Business Intelligence como Power BI o Tableau para diseñar un tablero de control (*dashboard*) interactivo con KPIs de volumen transaccional diario, montos promedio y patrones de depósitos/transferencias.



Referencias Bibliográficas

1. Kimball, R., & Ross, M. (2013). *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. John Wiley & Sons.
2. Microsoft Corporation. (2026). *SQL Server Documentation & Transactional Integrity Guidelines*. Microsoft Docs.